241080076 Vishwajeet Waghmare | 241080077 Awwab Wadekar
241080078 Yash Wadhwa | 241080082 Abiraj Kangotra

Hybrid Offline-First HMS: Project Estimation

5.Software Quality Assurance Plan (SQA plan)

Conceptual Foundations of Quality in Software Engineering

1. Introduction to Quality in Software Engineering

Software Quality Definition (IEEE 610.12): The degree to which a software system, component, or process meets specified requirements and customer/user expectations. In healthcare software systems, quality directly impacts clinical efficiency, data accuracy, and patient safety.

2. Quality Assurance (QA) vs. Quality Control (QC)

In software engineering, QA and QC are complementary yet distinct disciplines:

Dimension	**Quality Assurance (QA)**	**Quality Control (QC)**
Core Focus	**Process-Oriented:** Focuses on preventing defects by building and refining software development processes.	**Product-Oriented:** Focuses on identifying and fixing defects in the actual software binaries/builds.
Timing	**Proactive:** Executed continuously throughout the Software Development Life Cycle (SDLC).	**Reactive:** Executed during or after build phases via testing, inspections, and reviews.
Primary Goal	Ensure processes used to design and write software are effective, standardized, and compliant.	Verify that the final executable meets functional, security, and performance specifications.

Key Activities	Setting coding standards, process audits, CI/CD setup, tool selection, architecture reviews.	Unit testing, integration testing, boundary checks, bug logging, UI walkthroughs.
Responsibility	Whole team & process/quality leads.	Developers, test engineers, and peer reviewers.

3. The Cost of Quality (CoQ) Model

The Cost of Quality framework breaks down total quality investments into **Cost of Control** (conformance) and **Cost of Failure** (non-conformance):

A. Cost of Control (Conformance)

- **1. Prevention Costs:** Investments made to prevent defects before code is written.
    - Defining SQA processes, coding standards, and branch policies.
    - Setting up automated static analysis, linters, and CI/CD pipelines.
    - Team training on secure database handling and offline transaction design.
- **2. Appraisal Costs:** Expenditures associated with inspecting, measuring, and auditing code.
    - Formal peer code reviews and architectural walkthroughs.
    - Writing and maintaining automated unit, integration, and UI tests.
    - Running static code analysis tools (e.g., SonarQube, Flake8).

B. Cost of Failure (Non-Conformance)

- **1. Internal Failure Costs:** Expenses resulting from defects caught *before* deployment.
    - Debugging and refactoring failing unit or integration tests.
    - Re-testing bug fixes and rebuilding installers (.exe).
    - Database schema rework due to inadequate initial planning.
- **2. External Failure Costs:** Expenses resulting from defects discovered *after* deployment.
    - System crashes at hospital reception desks during peak patient check-in hours.
    - Data corruption or silent loss during offline-to-cloud synchronization.
    - Emergency hotfix deployments and severe reputational damage.

4. Application of SQA Principles in the Software Field

- **Shift-Left Testing:** Moving verification activities early into requirement and design phases drastically reduces defect resolution costs (resolving a bug during design costs ~1x vs. ~100x post-release).

- **Automated Defect Prevention:** Enforcing automated linting and formatting on git commits prevents entire classes of syntax, formatting, and type errors before code reviews occur.
- **Data-Driven Quality Metrics:** Tracking quantitative indicators—such as Defect Density, Test Pass Rates, Code Coverage Percentage, and Mean Time to Resolution (MTTR).

Software Quality Assurance (SQA) Plan (Part 1)

1. Project Context & Purpose

This SQA Plan establishes quality governance for the **Hybrid Offline-First Hospital Management System (HMS)**—developed by a 4-student engineering team over 3.5 months. Key artifacts include a Windows .exe desktop application, local SQLite database, embedded Raspberry Pi/ESP32 display nodes connected over LAN, and a FastAPI cloud synchronization server.

2. Software Quality Objectives

Quality Attribute	Target Metric / Criterion	Verification Method
High Performance & Latency	Local DB write/read queries complete in **<10ms**; instantaneous UI response.	Automated performance profiling logs on host PC.
Offline Reliability	**100% operational availability** when internet connection drops; zero UI freezes.	Network disruption simulation testing.
Data Integrity & Sync Accuracy	**Zero data loss** during background sync; 100% conflict resolution accuracy.	Automated sync transaction state verification.

Display Node Uptime	Embedded displays run continuously over local Wi-Fi without memory leaks or crashes.	72-hour continuous soak testing on Pi/ESP hardware.
Code Maintainability	**≥80% Unit Test Coverage** on core business logic; zero P1 linter warnings.	PyTest & Coverage reporting tools in automated build pipeline.

3. Governance Standards & Guidelines

- **IEEE 730-2014 Standard:** Software Quality Assurance Processes (tailored for lightweight agile delivery).
- **ISO/IEC 25010 Quality Model:** Guidance on Functional Suitability, Performance Efficiency, Usability, and Reliability.
- **Coding Conventions:** PEP 8 (Python) and Microsoft C# Guidelines enforced via automated linters (Flake8, Black).
- **Data Safety Principles:** SQLCipher local database encryption and secure HTTPS/TLS transport for cloud sync.

4. SQA Team Roles & Responsibilities (4-Developer Setup)

Role	Subsystem Assigned	SQA & Testing Responsibilities
Dev 1 (QA Lead / UI Lead)	Windows Desktop UI (.exe)	SQA Plan owner; UI automation, usability reviews, desktop issue tracking.
Dev 2 (Data Quality Lead)	Local SQLite DB & Cache	Database ACID compliance, schema validation, transaction integrity, encryption audits.

Dev 3 (Embedded Quality Lead)	Embedded Pi/ESP Display	LAN broadcast packet validation, embedded memory profiling, hardware soak testing.
Dev 4 (Sync Quality Lead)	Cloud API & Sync Engine	REST API contract testing, offline retry queue testing, edge-case conflict resolution.

5. Verification & Validation (V&V) Strategy

A. Static Verification Techniques

- **Peer Code Reviews:** Mandatory approval from at least one peer developer before merging pull requests into main.
- **Static Code Analysis:** Automated linting checks running on commit hooks to block messy code.
- **Design Walkthroughs:** Weekly reviews of DB migration scripts, local caching algorithms, and sync protocols.

B. Dynamic Validation Techniques

- **Unit Testing:** Automated unit tests covering isolated algorithms (e.g., token generation, stock balance math).
- **Integration Testing:** Testing desktop local DB operations combined with local LAN broadcasting to embedded displays.
- **System & Acceptance Testing:** End-to-end user journeys (e.g., reception patient check-in → live display token update).

Software Quality Assurance (SQA) Plan (Part 2)

1. Specific Testing Strategies for Key Subsystems

System Subsystem	Key Risks & Failure Modes	SQA & Testing Protocol
Desktop Application (.exe)	UI freezes during rapid reception data entry; unhandled invalid inputs.	Boundary value analysis, keyboard navigation testing, automated UI stress testing.
Local SQLite Database	Database file corruption during sudden system power loss or concurrent write locks.	Power-interruption simulations, Write-Ahead Logging (WAL) verification, transaction rollback checks.
Embedded Display Client	Screen freezes on Wi-Fi disconnect; memory leaks over multi-day continuous runtime.	Network reconnect auto-retry loops, socket packet corruption checks, 72-hour RAM profiling.
Cloud Sync Engine	Overwrite conflicts (e.g., same bed assigned offline on 2 PCs simultaneously).	Deterministic *Last-Write-Wins* timestamp verification, mock server offline/online transition tests.

2. Software Configuration Management (SCM) & Quality Control

- **Branching Model:** Protected main branch on GitHub. Feature development restricted to feature/* branches.
- **Semantic Versioning:** Release builds tagged using MAJOR.MINOR.PATCH format (e.g., v1.0.0-rc1).
- **Installer Build Verification:** Automated compilation of standalone .exe packages via PyInstaller/Inno Setup, verified on clean Windows virtual machines.

3. Defect Management & Severity Classification

Severity Level	Definition & Example	SLA Resolution Target	Release Gate Impact
P1: Critical	System crash, local DB corruption, patient data loss, offline mode failure.	**Fix within 24 Hours**	**Release Blocker:** Zero P1 bugs permitted.
P2: High	Display queue out of sync, medicine query error, cloud sync retry loop.	**Fix within 48 Hours**	**Release Blocker:** Zero P2 bugs permitted.
P3: Medium	UI layout misalignment on low-res screens, slow search filter response.	**Fix within 5 Days**	Max 3 permitted with documented workarounds.
P4: Low	Minor visual alignment issue, minor typo on doctor dashboard.	**Fix before final demo**	Non-blocking; deferred to post-MVP release.

4. SQA Milestone Audits & Quality Release Sign-Off

A. Audit Schedule

- **Audit 1 (End of Month 1):** Local DB schema stability audit, unit test coverage review (>70%), linter compliance.
- **Audit 2 (End of Month 2):** Local LAN broadcast protocol security review, embedded display soak test results.
- **Audit 3 (End of Month 3):** Sync conflict engine verification, disaster recovery & network drop testing.

B. Final Release Quality Sign-Off Criteria

Production deployment and demonstration sign-off require meeting all four criteria:

1. **Zero open P1 or P2 defects.**
2. **Automated unit test coverage ≥ 80%.**
3. **100% pass rate on network-disconnect offline simulation tests.**
4. **Clean, error-free execution of the standalone Windows .exe installer build.**

Conclusion

This document provides a complete Software Quality Assurance framework and project plan for the Hybrid Offline-First Hospital Management System. By combining core SQA principles, such as process-oriented defect prevention and the Cost of Quality framework, with actionable engineering controls, it establishes strict quality benchmarks. Through designated team roles, targeted subsystem testing protocols (covering SQLite integrity, desktop UI, embedded nodes, and cloud sync), and rigid P1/P2 release gates, the plan guarantees a resilient, zero-data-loss system optimized for reliable clinical operations.